

EX.No: 01	Study of command for Socket Programming	27.10.21
-----------	---	----------

AIM:-

To study the commands of the socket programming

Sockets:-

A Sockets is one end point of a two way communication link between two programs running on the network. Socket classes are used to present the connection between a client program and a server program.

Java. net Package provides two classes - socket and Server socket that implement the client side of the connection and the server side of connection respectively.

They connect to them on published ports when the server socket created it will register itself with the system as having an interest in client connection. construction for server socket having clients connection it can leave pending before it should be refers connections.

Port:

The TCP and UDP protocols use ports to map incoming data to a particular process running on a computer. Port numbers range from 0 to 65,535 because ports are represented by 16 bit numbers.

The port numbers ranging from 0-1023 are restricted. They are reserved for use by well known services such as HTTP, FTP and other system services.

TCP/IP Protocols:-

TCP provides a point to point channel for application that require reliable communication - The Hypertext Transfer protocol HTTP, file transfer protocol (FTP) and telnet are all examples of application that requires a reliable communication channel. The order in which data is sent and received over the network is critical to success of application. When HTTP is used to read from URL the data must be received in the order which it was sent HTML, file a ~~clip~~ file or some other information.

Stream Communication:

The stream communication protocol transfer mounted TCP is connection oriented programming protocol. In order, to communication over TCP protocol a



connection must first be established between the pair of sockets. While one of sockets listens for a connection request (server) other asks for a connection. Once two sockets have been connected they can be used to transmit data in both directions.

Creating Tcp services:

- (i) create a server socket attached to port number
`Socket server = new Socket (port);`
- (ii) wait for connections from clients requesting connection to port
- (iii) Get input and output streams associated with the socket

```
Out = new PrintWriter(channel, getOutput stream());  
Reader = new Input stream Reader (channel, get Input  
stream ());
```

```
in = new Buffered Reader (Reader);
```

```
String data = in. read line (), out. Print ("Hey!");
```

Creating Tcp clients:

- i) create a socket object attached to a remote host,
`Port socket client = new Socket (host, port);`
- ii) wait for connection from clients requesting connection to the port.

```
Socket channel = server accept();
```


iii) Get input and output stream associated with the socket

```
out = new PrintWriter(channel, getOutputStream());
```

```
reader = new InputStreamReader(channel, getInputStream());
```

```
in = new BufferedReader(reader);
```

```
String data = in.readLine(); out.println("Hey!");
```

Creating TCP services:-

(i) Create a socket object attached to a remote host,

```
Port socket client = new Socket(host, port);
```

Get input and output streams associated with the socket

```
out = new PrintWriter(client.getOutputStream());
```

```
reader = new InputStreamReader(client.getInputStream());
```

```
in = new BufferedReader(reader); String data =
```

```
in.readLine();
```

```
out.println("Hey!");
```


UDP:-

The UDP protocol consist of communication on that is not guaranteed between two application on the network, UDP is not connection based like TCP. It sends independent packets of data called datagram. from one application to another. sending datagram is much like sending a letter through the postal service. The order of delivery is not important and is not guaranteed and each message is independent of any other.

Datagram communication:-

Protocols user datagram protocols is a connection protocols, meanings that each time you send datagrams, you also need to send the local socket description and receiving sockets address.

Creating UDP server:-

(i) Create a datagram socket attached to a port

```
int port = 1234;
```



Datagram socket socket = new Datagram
socket (port);

Allocate space to hold the incoming packet &
create an instance of datagram packet
to hold the incoming data

byte [] buffer = new byte [1024];

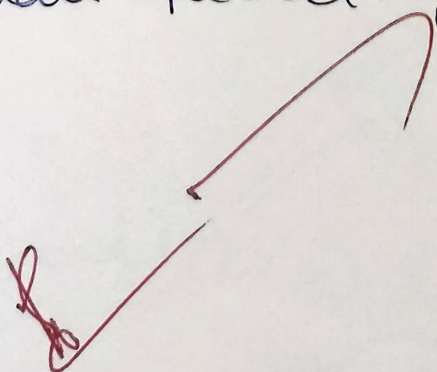
Datagram packet packet = new Datagram packet
(buffer, buffer, length);

(iii) Block until a packet is received then
extract the information you need from packet
socket.receive (packet);

int address remote Host = packet.get address();

int remote Port = packet.get port();

byte [] data = packet.get data();



Creating UDP clients:-

First allocate space to hold the data we are sending and create an instance of byte

```
[ ] buffer = new byte [1024]; int port =  
1234;
```

```
InetAddress host = InetAddress.getByName  
("magedlang.com");
```

```
Datagram packet p = new Datagram packet  
(buffer, buffer.length, host, port);
```

(ii) Create a datagram socket and send the packet using this socket.

```
Datagram socket socket = new Datagram  
socket();
```

```
socket.send(p);
```


--	--	--

`InetAddress hostName = socket.getLocalAddress();`

`int localPort = socket.getLocalPort();`

server socket construction & methods;

